

CONTEXT: PROJECT STRUCTURE

Below is the file tree. Use this to understand module relationships.

```
.
├── 1. 3month.md
├── 2. month.md
├── 3. weekly.md
├── 4. daily.md
├── 5. business_logic.md
├── 6. external_integrations.md
├── 7. analytic_capabilities.md
└── vibe_snapshot_env.txt
```

--- START_FILE: 1. 3month.md ---

Contains EVERYTHING on my radar: section, tasks and projects for each section.

Rewritten every three months to reprioritize tasks and remove dead weight tasks

It consulted when making the monthly list

--- END_FILE ---

--- START_FILE: 2. month.md ---

Is updated the end of every month. Consults the 3-month list

It is consulted in making the Weekly W list

Right beside each task, the late done date is recorded in this format [date, date

This helps to record the frequency of tasks, it is designed for low frequency tasks to be prioritized

The schedule section is consulted in making the Daily List

For each section we have tasks and then associated projects. Projects weigh more than task, the current default is around a 1.5 - 1 proportion

The tasks have assigned points based on relevance and complexity

Complex and Relevant: Normal point assigned

Complex and Non-relevant: High point (penalized)

This point is attached right next to the task and is expli

Each section has a ceiling point, School (14), Spiritual (10), DEPTS(3)..

This point is that which the total number points for this section must not exceed in a two active daily W-list

There is a schedule section contains a weekly schedule that contains tasks that have their occurrences set on a fixed days. In the app, the tasks should be automatically attached to their respective days

--- END_FILE ---

--- START_FILE: 3. weekly.md ---

Consults the monthly list.

Is consulted by the daily list

Each section is grouped and then ordered by the task priority level. Groups are Urgent, Mild, Other. All apply for the demanding sections, the first two suffice for the rest.

For tasks with subtasks it is written in special way to ensure
It is written in this format Task ^ [Subtask 1, Subtask 2]. The subtasks are written
right on top of them, and they are also assigned relevant scores

Each section is still capped by the 2-List ceiling point and it is clearly shown below
the tasks for easy reference by the daily list.

A new section, MISC is introduced here which serves to include urgent, spontaneous,
lightweight tasks that do not really fit in any of the sections.

Just as how the frequency of tasks is shown on the weekly list, the weekly list too
should show this
in the e version

--- END_FILE ---

--- START_FILE: 4. daily.md ---

A daily list consults the weekly list and the monthly list (well cos this help the
logging info for task)

and other valid daily lists

A valid daily list is referenced by other valid lists. There should be at most two valid
lists at a time

Each section is enclosed in parenthesis [], for balance. To show the subtasks, the
subtasks are written right on top of them, enclosed in parenthesis and they are also
assigned relevant scores written on top of them. The total number of points for all
tasks on the list is written at the bottom of the

The points for each section is calculated based on the task load and it is indicated
right below each section with a red pen. Tasks are assigned with the constraint of the
total number of points assigned to each section on the monthly list and the total points
for uncompleted tasks in a deferring list in this section. The aim is to ensure that the
total points in 2 lists do not exceed the ceiling point determined on the Monthly
W-list.

This list uses different symbols which mean different things

Checkmarks: To show completion

- W -> A completed task (literally called a W-list)

- A checkmark / half a W-> A half completed task (two checkmarks make a W)

- X -> A missed task

A key issue faced on the manual operation of this list is the inefficient representation
of the progress on a task especially with a really great multiplier or a task with
uneven number of points.

Attempts have been made to dynamically 'render' the checkmark in realtime: like to start
with a point, make a line, make it a checkmark, then draw another line downwards at a 60
degree, then draw another line up which completes the W.

Arrows: To show delegation

- -> -> This signifies a delegating a task to appear on the next W-list, and
completed(while the score ceiling constraints still apply), usually in cases of

emergency or overindulgence.

Underlining: To show deference

- ----- -> A line drawn under a task not completed before the next list is made (ideal should be 24hrs), it is considered as deferred task, more informally, a task on-the-line.
- -/-/-/-/- -> A crossed out line for on on-the-line task, it shows completion. Normal pencil lead for completion on the same day the list was closed, blue ink for completion on another day
- -/-/- -> Partial underlining and crossing out for incomplete tasks used to differentiate tasks not done at all.

A friction point I encountered with the manual operation of this list was accurately recording the date of completion for deferred tasks; logs for this are usually done once, and because a task is deferred it missed that opportunity and can either be done on that same day (after the audit), the next day, or even after that if an audit is not made. However, this should be solved by going e.

This list uses different colours to show emphasis

- Black -> Tasks and Sub-tasks, delegation(should change)
- Red -> Total points of a section, and points of each task
- Green -> To show multipliers
- Pencil Grey -> To show deference
- Deeper Pencil Grey Shaded -> To show completed deference on the day the list was closed
- Blue -> To show completed deference after the day the list was closed

How Long Is a list Valid?

A list should have a valid lifespan of 24hrs before it undergoes an audit. However, note that tasks have a valid lifespan of 48hrs. Uncompleted tasks after a list is audited make them tasks on-the-line indicating they have 24hrs of validity left.

The best results come when the audit is done at a specific time of day, usually adapted to match external natural rhythms e.g.,

How Use this System(2 full cycles)

- For the first W list, consult the Weekly list for priority tasks and the Monthly list for the history of log completion. Assign tasks to section while considering the ceiling point specified by the Monthly & Weekly List. Append subtasks on top of task in [], ensure the tasks and subtasks have their points right on top of them. Apply multipliers where necessary.

- After 24hrs, record all tasks with a W on the monthly log. All tasks with zero or incomplete completion are then underlined (full & partial). This list is now audited.

- To make a new list, the weekly list is consulted for grounding. The monthly log too is consulted; the aim is to prioritize low frequency tasks. The audited list is consulted considering two attributes of the uncompleted tasks

1 - The tasks themselves: A task with zero completion is forbidden from being on the new list except it is absolutely necessary. Incomplete tasks can be considered but the weight of their points is considered

2. The section they belong to: Note that the ceiling point generally applies to the sum of each section in two lists; so, this trailing point is noted and used to limit the tasks assignable to that section

- We now have 2 lists we are working were are working with consecutive. Note that there is an urgency stimulated to the audited list as they are only valid for 24hrs / the next audit where they will be marked a miss if not completed; the new list is stiill valid for 48hrs / 2 audits. To show the completion of a task on the audited list, the now underlined on-the-line task has its line crossed out a pencil with

- At the next audit, the audited list is prioritized, it has been marked in a way to reduce cognitive strain massively and point us directly to the defaulting tasks or now-completed tasks with the line drawn or crossed out. The now-completed tasks are recorded while the others are marked with a miss. The audited list is now closed.

- The current list is now audited

- A new list is then made

What is a weekly audit?

- All lists in a week are used for a weekly assessment primarily to compare to last weeks established metrics. Perecentage completion (total, section), Total points etc. are noted

--- END_FILE ---

--- START_FILE: 5. business_logic.md ---

It must be at the heart a planner first, then and execution engine. We need to make the major Yearly Goals and then the W-Lists.

It must mirror the workflow of the physical lists as much as possible. Especially the intereferencing of lists when we need to make lists. We should be able to have multiple kinds of lists visible and interactable on the working interface.

The colour coding of icons and symbols is non-negotiable

The layout, the way tasks are arranged also is non-negotiable. The differentitation of tasks from projects too.

The 3-month list should have tasks that persist throughout all lists, like they should be auto-completed when you wanna type them in other lists.

Bulk assignment of scores, sections, tags. It must be a power app

This should not just be built for the happy path alone, we should be able to manually workout backdate completion. The system also has some other completion based components like the Reinforced Spaced Repitition that absolutely need this. We need to handle backlogs and failure gracefully

--- END_FILE ---

--- START_FILE: 6. external_integrations.md ---

Pomodoro System: I mentioned this being used as a tie breaker or an alternative grading system to be used to designate completion points to tasks. I implemented a system where 1 full custom pomodoro of a task was worth 1 point in the W list. Meaning for academic tasks 23 minutes = 1 W point.

Pink Slip: - This splits every course by lecturer down to artificial blocks I divide each section into. I then track every subsection by date read, so I can identify areas not yet read and then focus on them. The app will have the capability of dividing tasks into children that make a whole, and then track these individual parts, showing their last date maybe on a dedicated page

Reinforced Spaced Repetition: This is the pink slip system in action, I adopt the spaced scientifically proven system of intermittent revision theory which is said to improve retention. We should be able to toggle the RSR mode on for some tasks and so make them auto-delegated into the W List.

External Forced Schedules / Calendar: What I realized as the future of the W-List is to it so aligned with your external schedule (the schedule at the end of the existing monthly list is a fantastic prototype) that these tasks are automatically integrated into your schedule on their designated days while still enforcing your point ceiling constraints reducing further unnecessary assignments.

Day Audit Report: We should be able to see tasks completed in an organized form ready for export and accountability. Can we experiment with cycles, one for the audit, and one for the closing

Cloud and Interdevice Sync: I think it would be really cool to have me sync my current progress my phone and my laptop through a free platform like One Drive to ensure everything goes smoothly.

--- END_FILE ---

--- START_FILE: 7. analytic_capabilities.md ---

I was extremely negatively constrained in my existing implementation of the W-list as everything I did was done manually. I think this will be one of the most important leverage points as we go electronic. We need an analytic system as sophisticated as the engineering system that has been built

- Section, subsection, tag analysis
- Forecast
- Completion rate for various Section
- Most delegated, missed
- Timeframe analysis

AND MANY MORE to take advantage of our going e!

--- END_FILE ---

--- START_FILE: vibe_snapshot_env.txt ---

VIBECODE SNAPSHOT ENVIRONMENT

Generated: 2026-03-24T10:23:19.271890

Platform: Windows 10

Python Version: 3.12.10

Executable: C:\Users\AY\Desktop\APPS\vibecode-project\.venv\Scripts\python.exe

annotated-types==0.7.0

anyio==4.12.1

attrs==25.4.0

brrotli==1.2.0

certifi==2026.1.4

cffi==2.0.0

charset-normalizer==3.4.4

click==8.3.0

colorama==0.4.6

cryptography==46.0.4

cssselect2==0.8.0

defusedxml==0.7.1

distro==1.9.0

fonttools==4.60.1

fpdf2==2.8.5

google-ai-generativelanguage==0.6.15

google-api-core==2.29.0

google-api-python-client==2.189.0

google-auth==2.49.0.dev0

google-auth-httpplib2==0.3.0

google-auth-oauthlib==1.2.4

google-genai==1.62.0

google-generativeai==0.8.6

googleapis-common-protos==1.72.0

grpcio==1.78.0

grpcio-status==1.71.2

h11==0.16.0

httpcore==1.0.9

httplib2==0.31.2

httpx==0.28.1

httpx-sse==0.4.3

idna==3.11

jaraco.classes==3.4.0

jaraco.context==6.1.0

jaraco.functools==4.4.0

jiter==0.13.0

jsonschema==4.26.0

jsonschema-specifications==2025.9.1

keyring==25.7.0

Markdown==3.10.1

markdown-it-py==4.0.0

mcp==1.26.0

mdurl==0.1.2

mistune==3.1.4

```
more-itertools==10.8.0
oauthlib==3.3.1
openai==2.17.0
pathspec==0.12.1
pillow==12.0.0
proto-plus==1.27.1
protobuf==5.29.6
pyasn1==0.6.1
pyasn1_modules==0.4.2
pycparser==2.23
pydantic==2.12.5
pydantic-settings==2.12.0
pydantic_core==2.41.5
pydyf==0.11.0
Pygments==2.19.2
PyJWT==2.11.0
pyparsing==3.3.2
pypdf==6.6.2
pyphen==0.17.2
PyQt6==6.10.1
PyQt6-Qt6==6.10.1
PyQt6_sip==13.10.3
PySimpleGUI==5.0.10
python-dotenv==1.2.1
python-multipart==0.0.22
pywin32==311
pywin32-ctypes==0.2.3
PyYAML==6.0.3
referencing==0.37.0
requests==2.32.5
requests-oauthlib==2.0.0
rich==14.2.0
rpds-py==0.30.0
rsa==4.9.1
shellingham==1.5.4
sniffio==1.3.1
sse-starlette==3.2.0
starlette==0.52.1
tenacity==9.1.4
tinycss2==1.4.0
tinyhtml5==2.0.0
tqdm==4.67.3
typer==0.20.0
typing-inspection==0.4.2
typing_extensions==4.15.0
uritemplate==4.2.0
urllib3==2.6.3
uvicorn==0.40.0
# Editable install with no version control (vibecode==1.0.0)
-e c:\users\ay\desktop\vibecode-project
weasyprint==66.0
webencodings==0.5.1
websockets==15.0.1
```

zopfli==0.2.3.post1

--- END_FILE ---

